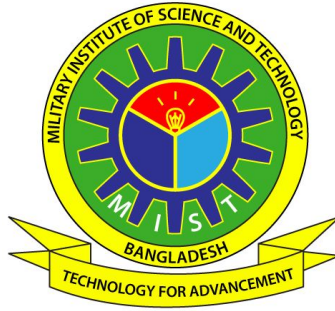


MILITARY INSTITUTE OF SCIENCE AND TECHNOLOGY



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSE 364: SOFTWARE DEVELOPMENT PROJECT-I

SOFTWARE REQUIREMENTS SPECIFICATION DOCUMENT

of

Local Volunteer Radar

A Community Volunteer Matching Web Platform

Group: A3

Batch: CSE-23 **Section:** A

Submitted By

Submitted To

Md. Iftehajul Hoque	ID: 202314016	Lt Col Md Abul Basar
Sheikh Mohammad Shakib	ID: 202314029	Prof Dr. Md. Ashraful Islam
Md. Arif Hossain Joy	ID: 202314043	Asso. Prof. Dr. T M Shahriar Sazzad
Shihab Ahmed	ID: 202314049	Lec Alisha Kabir
Wasif Darain	ID: 202314053	Lec Sanjana Jahan Sifat

Submission Date: March 8, 2026

Contents

1	Preface	1
1.1	Problem Background	1
1.2	Motivation	1
1.3	Limitations of Existing Work	2
2	Introduction	2
2.1	Purpose	2
2.2	Intended Stakeholders	3
2.3	Scope	3
2.3.1	The system does:	3
2.3.2	The system does not:	3
3	Glossary	4
4	Requirements Discovery	7
4.1	Literature Review: Existing Systems and Gaps	7
5	User Requirements	8
5.1	Volunteer Requirements	8
5.2	Organizer Requirements	9
5.3	Administrator Requirements	9
5.4	Matching & Location Requirements	9
6	System Architecture	10
6.1	Architecture Diagram	10
6.2	Justification for Type of Architecture	10
7	System Requirements Specification	11
7.1	Functional Requirements	12
7.1.1	Volunteer Module	12

7.1.2	Organizer Module	12
7.1.3	Admin Module	12
7.2	Non-Functional Requirements	13
7.2.1	Performance Requirements	13
7.2.2	Security Requirements	13
7.2.3	Usability Requirements	13
7.2.4	Reliability Requirements	13
7.2.5	Scalability Requirements	13
7.2.6	Maintainability Requirements	14
7.3	Requirements Classification	14
8	System Model	18
8.1	Context Diagram	18
8.2	Use Case Diagram	19
8.3	Activity Diagrams	23
9	Appendix	26
10	Conclusion	27
11	References	28

List of Tables

1	Mapping between functional and non-functional requirements across system categories	14
2	Literature review and comparison against Local Volunteer Radar	26

List of Figures

1	System architecture diagram	10
2	Context diagram for Local Volunteer Radar	18

3	Use case diagram for Volunteer module	20
4	Use case diagram for Organizer module	21
5	Use case diagram for Admin module	22
6	Activity diagram for Admin Module	23
7	Activity diagram for Volunteer Module	24
8	Activity diagram for Organizer Module	25

1 Preface

1.1 Problem Background

One of the most common problems in community volunteering is that people who want to help simply cannot find the right opportunity, and event organizers that need help cannot find the right people. Local events and organizers (such as NGOs) often struggle to get volunteers on short notice, and there is no single place where someone can go to see what volunteer activities are happening nearby.

On top of that, many skilled volunteers: people who can teach, provide first aid, handle logistics, or support media, go unnoticed because no system exists to match them with tasks that suit their abilities. The result is that a lot of willing and capable people never end up contributing, while organizers continue to be understaffed.

The way volunteers are currently recruited and registered also tends to be slow and manual: relying on phone calls, paper forms, or scattered social media posts. This wastes time for everyone involved, especially in situations where a fast response is needed.

1.2 Motivation

For students and young professionals, it offers real-world experience in teaching, leadership, communication, and problem-solving before they enter the workforce. Working with organizers also creates a sense of purpose and connection to the community.

However, these benefits are out of reach for many people simply because finding and joining a volunteer activity is too complicated. Local Volunteer Radar aims to fix that by making the whole process easier: helping volunteers find nearby opportunities that match their skills, and helping organizers fill their needs quickly and efficiently.

1.3 Limitations of Existing Work

Existing volunteer management tools and academic systems have made some progress in this space, but they consistently fall short in a few key areas. Most platforms are too broad in scope: they list opportunities across large regions but are not built to surface urgent, nearby needs or match volunteers based on location and specific skills. On the research side, systems that have been developed tend to suffer from technical limitations such as poor scalability or remaining as frontend prototypes with no working backend. Across the board, the same gaps keep appearing: no smart matching, no real-time local awareness, and no complete end-to-end solution. Local Volunteer Radar is designed to address all of these directly.

2 Introduction

2.1 Purpose

Volunteering is an important part of how communities function, but the process of connecting volunteers with organizers is often inefficient. People who want to help struggle to find relevant opportunities nearby, skilled volunteers are frequently overlooked because there is no system to match them properly, and organizers spend too much time on manual recruitment and coordination instead of focusing on their actual work.

Local Volunteer Radar addresses this by providing a centralized web platform where volunteers and organizers can connect more effectively. Volunteers can create a profile, list their skills and availability, and browse or get matched to nearby opportunities. Organizers can create events with specific requirements, review and approve applicants, and view their volunteer activities from a single dashboard. An admin layer sits above both to oversee platform activity and maintain quality control.

The platform uses location data and skill-based matching to make sure volunteers are shown opportunities that are actually relevant to them, rather than having to search through listings that do not apply. Communication tools are built in so that both volunteers and organizers stay

informed throughout the process.

The goal is to reduce the time and effort it takes for volunteers to find meaningful work and for organizers to find the right people, so that more community activity can actually happen.

2.2 Intended Stakeholders

- **Volunteers:** Individuals who want to contribute their time and skills to local causes and need a straightforward way to find and join relevant activities near them.
- **Organizers:** Community groups, charities, and event organizers who need to recruit the right volunteers quickly and manage their activities efficiently.
- **Administrator:** Responsible for overseeing the system, approving events, and ensuring the platform runs smoothly and reliably.

2.3 Scope

2.3.1 The system does:

- Volunteer registration and skill-based profile management
- Location-aware discovery of nearby volunteer opportunities
- Matching of volunteers to tasks based on skills
- Event posting and volunteer recruitment tools for organizers
- Admin dashboard for platform oversight and analytics

2.3.2 The system does not:

- Handle financial transactions or paid work arrangements
- Issue formal volunteer certifications or accreditation
- Replace the internal management systems of organizations

3 Glossary

Volunteer An individual who registers on the platform to offer their time and skills to community events and activities. Volunteers can browse opportunities, apply for tasks, and track their participation history through a personal dashboard.

Organizer A community group, organizer, or event coordinator that registers on the platform to post volunteer opportunities, define task requirements, manage applications, and communicate with accepted volunteers.

Administrator (Admin)

The highest-privileged user role in the system, responsible for approving or rejecting newly registered organizers and submitted events, managing all accounts on the platform, and overseeing system-wide analytics.

Skill Category

A predefined label assigned to a volunteer's profile and to event requirements, used by the matching system to connect volunteers to relevant opportunities. Examples include teaching, first-aid, logistics, media, and distribution.

Skill-Based Matching

The process by which the system recommends volunteer opportunities to a user based on the overlap between the skills listed on their profile and the requirements specified by an organizer for a given task.

Event / Task

A volunteer opportunity posted by an organizer that includes details such as required skills, location, time window, and the number of volunteers needed. Events must be approved by the Admin before becoming publicly visible.

Interactive Map

A location-aware interface powered by the Google Maps API that displays nearby

volunteer events relative to a user's current position, allowing geographic browsing and filtering.

Dashboard

A role-specific interface provided to each user type. Volunteers use it to view their application status, history, and announcements. Organizers use it to manage their events and volunteer lists. Admins use it to monitor platform-wide activity and analytics.

Announcement

A broadcast message sent by an organizer to all volunteers registered for a specific event, used to communicate updates, changes, or important information related to that activity.

Real-Time Notification

An automated alert delivered to a volunteer via email or SMS when a relevant event occurs, such as application approval, event update, or organizer announcement.

AI Copilot An integrated AI assistant powered by the Groq LLaMA 3.1 model, used by organizers to automatically generate event descriptions and promotional content, and available to volunteers for guidance and platform assistance.

Google OAuth

A third-party authentication mechanism that allows users to register and log in securely using their existing Google account credentials, eliminating the need for a separate username and password.

Google Calendar API

An external service integration that allows the system to automatically add a volunteer's approved events to their personal Google Calendar upon registration for those events.

Cloudinary

A cloud-based media management platform used to store and serve uploaded files,

including volunteer and organizer profile pictures and any event-related images.

Nodemailer

A Node.js module used to send automated emails, including password reset links and platform notifications, via a configured SMTP service.

Socket.IO A JavaScript library enabling real-time, bidirectional communication between the server and connected clients. Used in Local Volunteer Radar for live messaging between volunteers and organizers.

Role-Based Access Control (RBAC)

A security mechanism that restricts system features and data access based on the authenticated user's role: Volunteer, Organizer, or Admin, ensuring each user can only perform actions appropriate to their role.

MongoDB A NoSQL document-oriented database used to persist all system data, including user accounts, event records, chat messages, volunteer applications, and organizer profiles.

React + Vite

The frontend technology stack used to build the user interface. React provides the component-based UI framework, while Vite serves as the build tool for fast development and optimized production output.

Node.js + Express

The backend runtime and web framework used to handle all server-side application logic, including authentication, event management, AI integration, email dispatch, and API routing.

4 Requirements Discovery

4.1 Literature Review: Existing Systems and Gaps

A widely used volunteer platform is discussed by VolunteerMatch (2023) where the emphasis is placed on profile-based filtering and keyword search to connect volunteers with available opportunities. The platform allows organisations to publish volunteer roles while individuals create personal profiles describing their interests, availability, and previous experience. Although this approach enables quick discovery of opportunities and supports a large network of nonprofit organisations, the system does not enforce structured skill declarations or formal role separation between volunteers and organisers. As a result, the matching process largely depends on self-reported information and manual screening by organisations, which may lead to impersonation risks, mismatched placements, and inefficient utilisation of volunteer capabilities, particularly when large numbers of volunteers apply for the same opportunity [1]. A complementary platform is reviewed by Idealist.org (2023), which centralises nonprofit job and volunteer listings across a wide range of organisations and sectors. The platform primarily functions as a directory where organisations advertise opportunities and interested volunteers browse listings based on categories, causes, or geographic locations. However, all application processing is handled externally by the respective organisations, meaning that the platform itself does not manage volunteer onboarding or communication. This approach reduces transparency and administrative efficiency within the system, as there is no built-in approval workflow, activity tracking mechanism, or analytics layer to monitor volunteer participation and organisational engagement [2]. An aggregation-based approach is examined by Points of Light (2023) through the All for Good platform, which collects volunteer opportunities from multiple partner sources and presents them through a unified interface that supports keyword search, category filters, and map-based discovery tools. While this aggregation model increases visibility of volunteer opportunities and simplifies discovery for users, volunteers are typically redirected to external websites to complete registration or application procedures. Consequently, the volunteer journey becomes fragmented across multiple systems, preventing the platform from offering unified lifecycle management features such as centralised onboarding, progress tracking, participation

history, and post-event feedback mechanisms [3]. On the research side, a recommendation model is proposed by Zhang and Chen (2019) in which collaborative filtering techniques are applied using historical volunteer participation data to recommend opportunities to users with similar interests and behavioural patterns. Although this data-driven method improves recommendation relevance for returning users with established activity histories, it suffers from the well-known cold-start problem when new users join the platform or when newly created opportunities lack sufficient interaction data. Furthermore, the model does not incorporate explicit skill declarations, structured competency profiles, or geospatial filtering, which are often important factors when assigning volunteers to appropriate roles in real-world environments [4]. A geospatial solution is presented by Rahman and Islam (2020) where geographic information systems are used to perform radius-based matching and skill classification to support volunteer deployment in disaster response situations. The system prioritises geographic proximity and relevant skill categories in order to rapidly allocate volunteers to urgent tasks during emergencies. However, the model was designed specifically for emergency response contexts and therefore lacks several capabilities required for a general-purpose volunteer management platform, such as comprehensive administrative controls, user verification mechanisms, structured approval workflows, and long-term volunteer engagement management tools [5].

The information extracted through this literature review is presented in a structured comparative format in Table 2 in Appendix, highlighting the specific gaps that Local Volunteer Radar is designed to address.

5 User Requirements

5.1 Volunteer Requirements

- The system shall allow volunteers to create a personal profile listing their skills (e.g., teaching, first-aid, distribution, media, logistics etc.)
- The system shall allow volunteers to join an event with a single click
- The system shall provide each volunteer with a personal dashboard showing their history of

completed volunteering activities

5.2 Organizer Requirements

- The system shall allow organizers to create and publish volunteer tasks specifying requirements, location, time, and required skill categories
- The system shall enable organizers to review and approve or reject incoming volunteer requests
- The system shall allow organizers to monitor volunteer participation and track task progress in real time
- The system shall provide organizers with an announcement tool to broadcast communications to registered volunteers

5.3 Administrator Requirements

- The system shall allow administrators to manage all registered volunteer and organizer accounts on the platform
- The system shall provide administrators with a global analytics dashboard displaying platform-wide data and trends
- The system shall require administrator approval before newly submitted events are made publicly visible

5.4 Matching & Location Requirements

- The system shall recommend volunteer opportunities to users based on their stated skills and location

6 System Architecture

6.1 Architecture Diagram

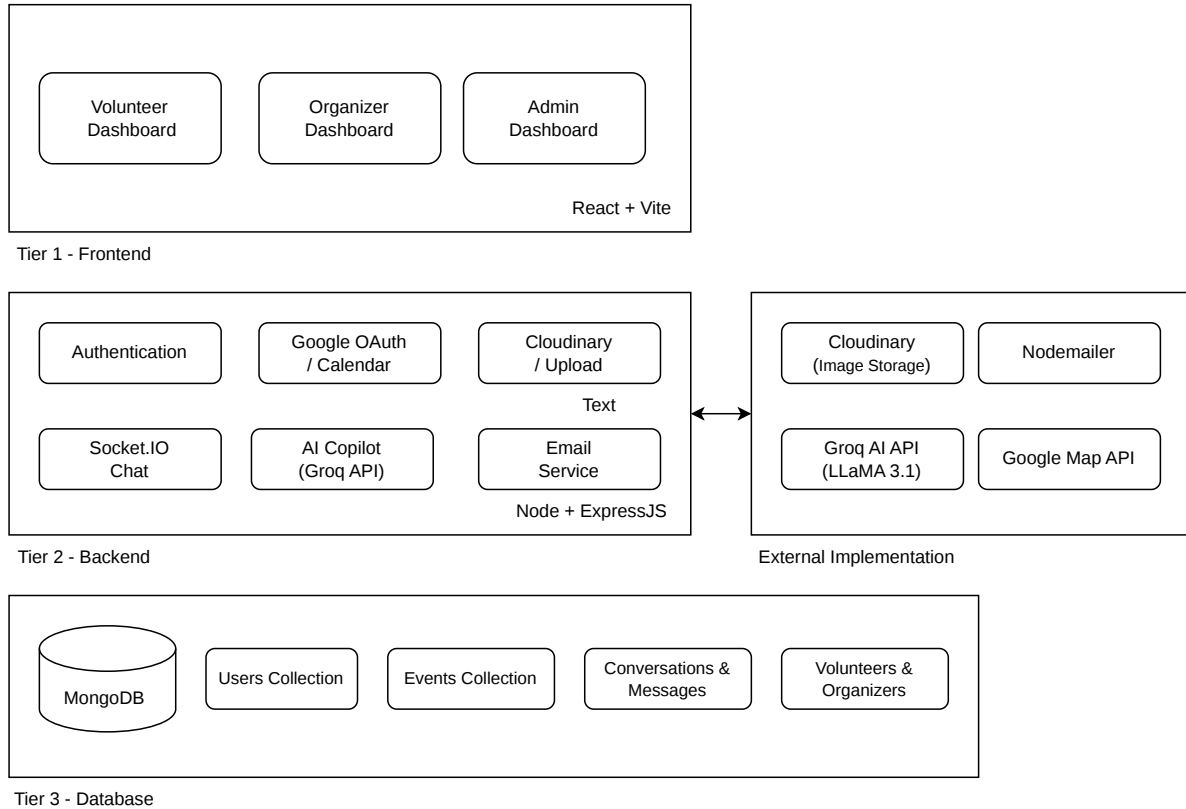


Figure 1: System architecture diagram

6.2 Justification for Type of Architecture

Local Volunteer Radar is built on a three-tier client-server architecture, a design choice that reflects the practical needs of a multi-role, real-time web platform.

Frontend Tier. Developed using React and Vite, this tier is responsible entirely for the user interface and experience. It is divided into three role-specific dashboards, one each for Volunteers, Organizers, and Admins, ensuring that every user sees only the controls and data relevant to their role. React’s component-based model makes it straightforward to maintain these distinct interfaces without duplicating logic, while Vite ensures fast build times and efficient asset delivery.

Backend Tier. Built with Node.js and Express, the backend serves as the central processing layer and the sole point of contact between the frontend and all external services. It handles user authentication, event and profile management, role-based access enforcement, real-time messaging via Socket.IO, AI copilot interactions through the Groq API, and outbound email through Nodemailer. Centralizing this logic in a single server layer means that security rules, business logic, and API integrations are applied consistently regardless of which frontend role triggers them.

Database Tier. MongoDB is used to store all persistent data, including user accounts, event records, volunteer applications, chat conversations, and organizer profiles. MongoDB's document model suits this platform well because the core data entities, i.e., events, profiles, and messages are naturally variable in structure and do not benefit from a rigid relational schema.

External Service Isolation. All third-party integrations: Cloudinary for media storage, Google OAuth for authentication, Google Maps API for location features, Google Calendar API for event scheduling, and the Groq AI service for content generation, are handled exclusively at the backend tier. This isolation ensures that any change to an external provider's API requires only a backend update, with no impact on the frontend.

This layered separation of concerns makes each component independently maintainable and testable, supports horizontal scaling as user load grows, and provides a clear boundary for security enforcement. It is the appropriate architecture for a platform of this scope that must balance real-time interactivity, multi-role access control, and integration with multiple third-party services.

7 System Requirements Specification

The system requirements define the services provided by the Local Volunteer Radar system and the constraints under which it must operate. Requirements are divided into functional and

non-functional requirements.

7.1 Functional Requirements

7.1.1 Volunteer Module

- **FR-1** The system shall allow volunteers to register and create an account.
- **FR-2** The system shall allow volunteers to log in and log out securely.
- **FR-3** The system shall allow volunteers to select and update skill categories (e.g., teaching, first-aid, logistics, media).
- **FR-4** The system shall allow volunteers to search and filter events by skill.
- **FR-5** The system shall allow volunteers to join or withdraw from a volunteer task.
- **FR-6** The system shall provide a personal dashboard showing current and past volunteering activities.
- **FR-7** The system shall allow volunteers to view event details including requirements, location, time, and organizer information.

7.1.2 Organizer Module

- **FR-8** The system shall allow organizers to register and create organizational profiles.
- **FR-9** The system shall allow organizers to create and post volunteer tasks.
- **FR-10** The system shall allow organizers to define task requirements including skill category, time, and location.
- **FR-11** The system shall allow organizers to approve or reject volunteer applications for an event.
- **FR-12** The system shall allow organizers to post announcements related to events.
- **FR-13** The system shall support a chat system between volunteers and organizers.

7.1.3 Admin Module

- **FR-14** The system shall allow administrators to manage volunteer and organizer accounts.
- **FR-15** The system shall allow administrators to approve or reject posted events.

- **FR-16** The system shall provide global data analytics dashboards.

7.2 Non-Functional Requirements

7.2.1 Performance Requirements

- **NFR-1** The system shall respond to user actions within 3 seconds under normal load.
- **NFR-2** The system shall support at least 100 concurrent users.

7.2.2 Security Requirements

- **NFR-3** The system shall implement secure authentication and encrypted password storage.
- **NFR-4** The system shall enforce role-based access control (Volunteer, Organizer, Admin).
- **NFR-5** The system shall protect user data in compliance with applicable data privacy standards.

7.2.3 Usability Requirements

- **NFR-6** The system shall provide a responsive user interface compatible with PC browsers.
- **NFR-7** The system shall provide intuitive navigation with minimal training required.

7.2.4 Reliability Requirements

- **NFR-8** The system shall achieve 95% uptime excluding scheduled maintenance.
- **NFR-9** The system shall perform automated daily database backups.

7.2.5 Scalability Requirements

- **NFR-10** The system shall support horizontal scaling to accommodate increasing user growth.

7.2.6 Maintainability Requirements

- **NFR-11** The system shall follow modular architecture to allow independent component updates.
- **NFR-12** The system shall maintain structured documentation for future development.

7.3 Requirements Classification

Table 1: Mapping between functional and non-functional requirements across system categories

Sl. No.	Category Description	Functional	Non-functional	Remarks	
				Functional Requirements	Non-Functional Requirements
1	User Registration & Authentication	✓	✗	FR-1 (Register volunteer), FR-2 (Login/Logout), FR-9 (Register organizer)	NFR-3 (Secure authentication), NFR-4 (Role-based access control)
2	Profile & Skill Management	✓	✗	FR-3 (Select/update skills), FR-6 (Personal dashboard)	NFR-1 (Response time ≤ 3 sec), NFR-6 (Responsive interface)

Sl. No.	Category Description	Functional	Non-functional	Remarks	
				Functional Requirements	Non-Functional Requirements
3	Event Discovery	✓	✗	FR-4 (Search/filter events), FR-7 (View event details)	NFR-1 (Performance response time), NFR-10 (Scalability support)
4	Volunteer Participation	✓	✗	FR-5 (Join/Withdraw events)	NFR-8 (Uptime reliability), NFR-9 (Database backup)
5	Notifications & Communication	✓	✗	FR-11 (Post announcements), FR-12 (Chat system)	NFR-1 (Notification latency), NFR-3 (Secure communication)
6	Organization Task Management	✓	✗	FR-8 (Create tasks), FR-9 (Define requirements), FR-10 (Approve/reject volunteers)	NFR-4 (Access restrictions), NFR-11 (Modular architecture)

Sl. No.	Category Description	Functional	Non-functional	Remarks	
				Functional Requirements	Non-Functional Requirements
7	Progress Tracking & History	✓	✗	FR-6 (Volunteer history dashboard)	NFR-5 (Data protection compliance), NFR-9 (Backup & integrity)
8	Admin Management	✗	✓	FR-13 (Manage users), FR-14 (Event approval), FR-15 (Analytics dashboard)	NFR-4 (Restricted admin privileges), NFR-8 (High availability)
9	Chat System	✗	✓	FR-12 (Chat between volunteers and organizers)	NFR-3 (Message encryption), NFR-8 (System availability)
10	Overall System Performance	✗	✓	—	NFR-1 (Response time ≤ 3 sec), NFR-2 (≥ 100 concurrent users)

Sl. No.	Category Description	Functional	Non-functional	Remarks	
				Functional Requirements	Non-Functional Requirements
11	Security	✗	✓	—	NFR-3 (Encrypted passwords), NFR-4 (Role-based control), NFR-5 (Privacy compliance)
12	Maintainability & Scalability	✗	✓	—	NFR-10 (Horizontal scalability), NFR-11 (Modular design), NFR-12 (Structured documentation)

8 System Model

The system model of Local-Volunteer-Radar is presented through three diagram types: a context diagram that defines the system boundary, a use case diagram that captures the functional scope from the perspective of each actor, and an activity diagram that illustrates the flow of key operations within the system.

8.1 Context Diagram

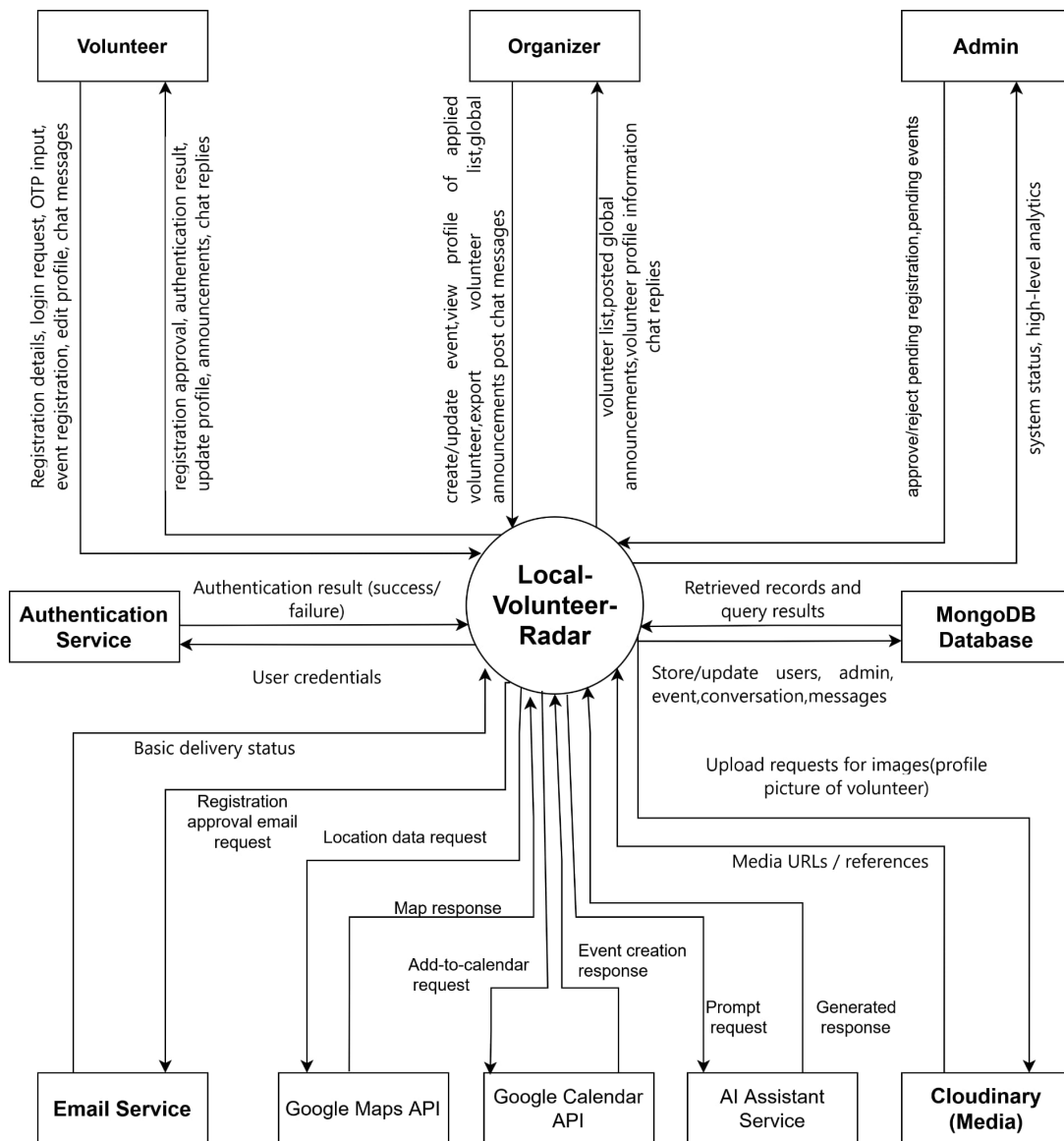


Figure 2: Context diagram for Local Volunteer Radar

The context diagram (Figure 2) depicts Local-Volunteer-Radar as a centralized platform interfacing with multiple external entities. Volunteers register, log in, submit event applications, send chat messages, and interact with the AI assistant, receiving event listings, application updates, chat responses, calendar confirmations, and guidance. Organizers manage events, review applications, communicate via real-time chat, and monitor participation analytics. The Admin manages organizer accounts and system configurations.

On the service side, the Authentication Service handles credential verification and token validation, while the Email Service sends confirmations and notifications. MongoDB stores all data: user profiles, events, applications, participation history and chat logs, returning results on query.

External integrations include the Google Maps API for event location visualization, the Google Calendar API for creating personal calendar events, and the AI Assistant Service for processing user prompts and delivering intelligent guidance.

8.2 Use Case Diagram

The use case diagrams in Figures 3, 4 and 5 identify three primary actors: Volunteer, Organizer and Admin, respectively, alongside external actors, the Google Maps API, Google Calendar API, and Email Service. Volunteers can register or log in, browse and apply for events, send chat messages, interact with the AI assistant, view application status, and manage their profiles. Organizers log into the dashboard to create and manage events, review volunteer applications, communicate via chat, and monitor participation analytics. The Admin holds exclusive access to manage organizer accounts and system-level configurations. External services support key operations: the Google Maps API provides event location visualization, the Google Calendar API handles personal calendar event creation, and the Email Service manages notifications and confirmations.

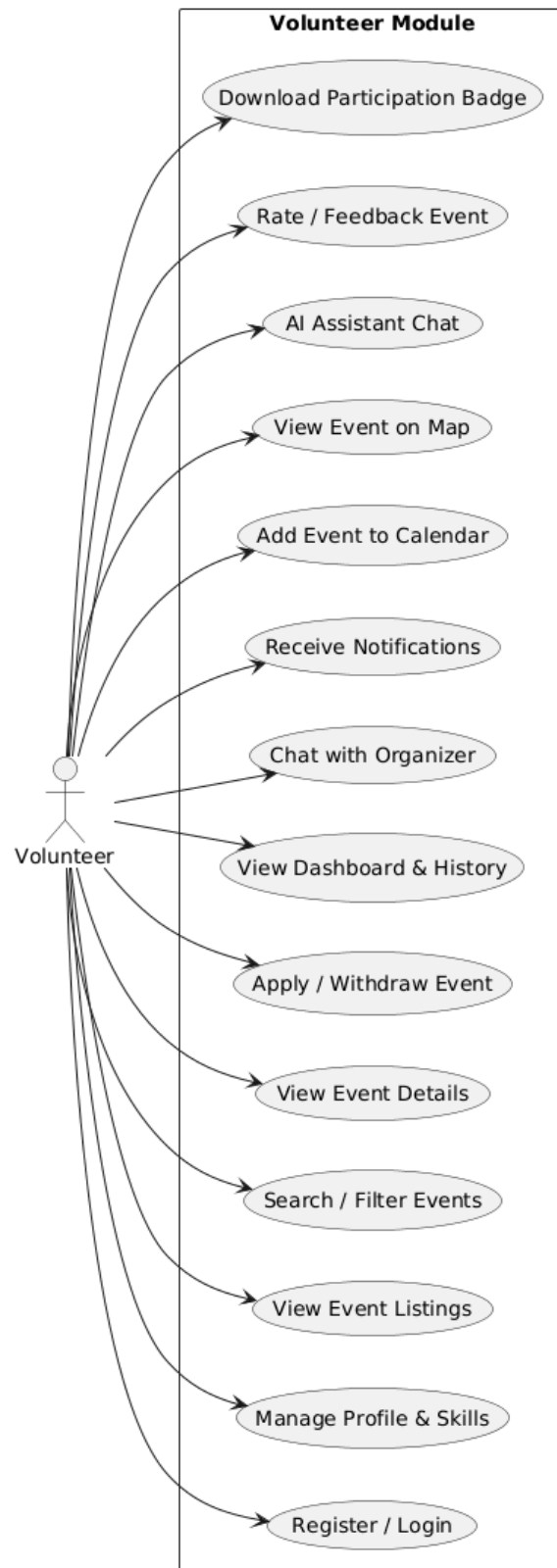


Figure 3: Use case diagram for Volunteer module

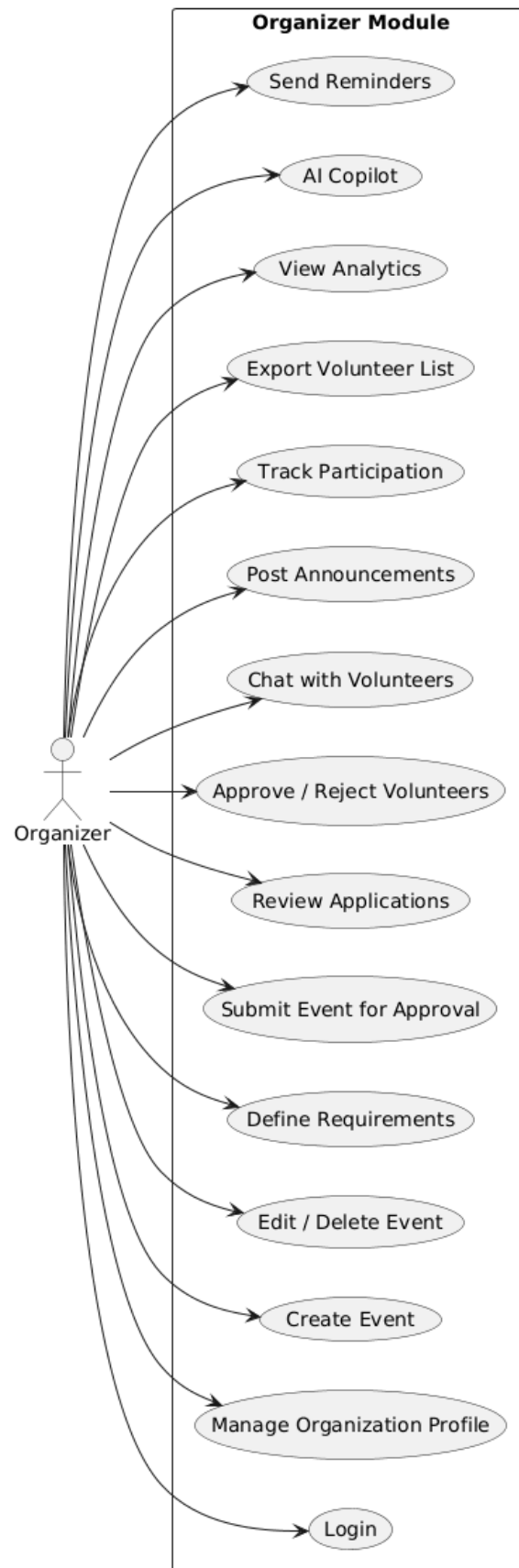


Figure 4: Use case diagram for Organizer module

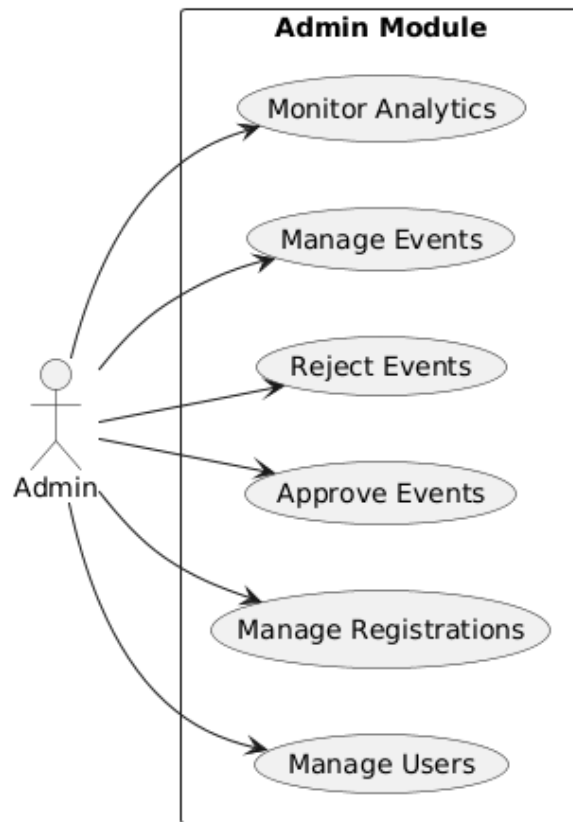


Figure 5: Use case diagram for Admin module

8.3 Activity Diagrams

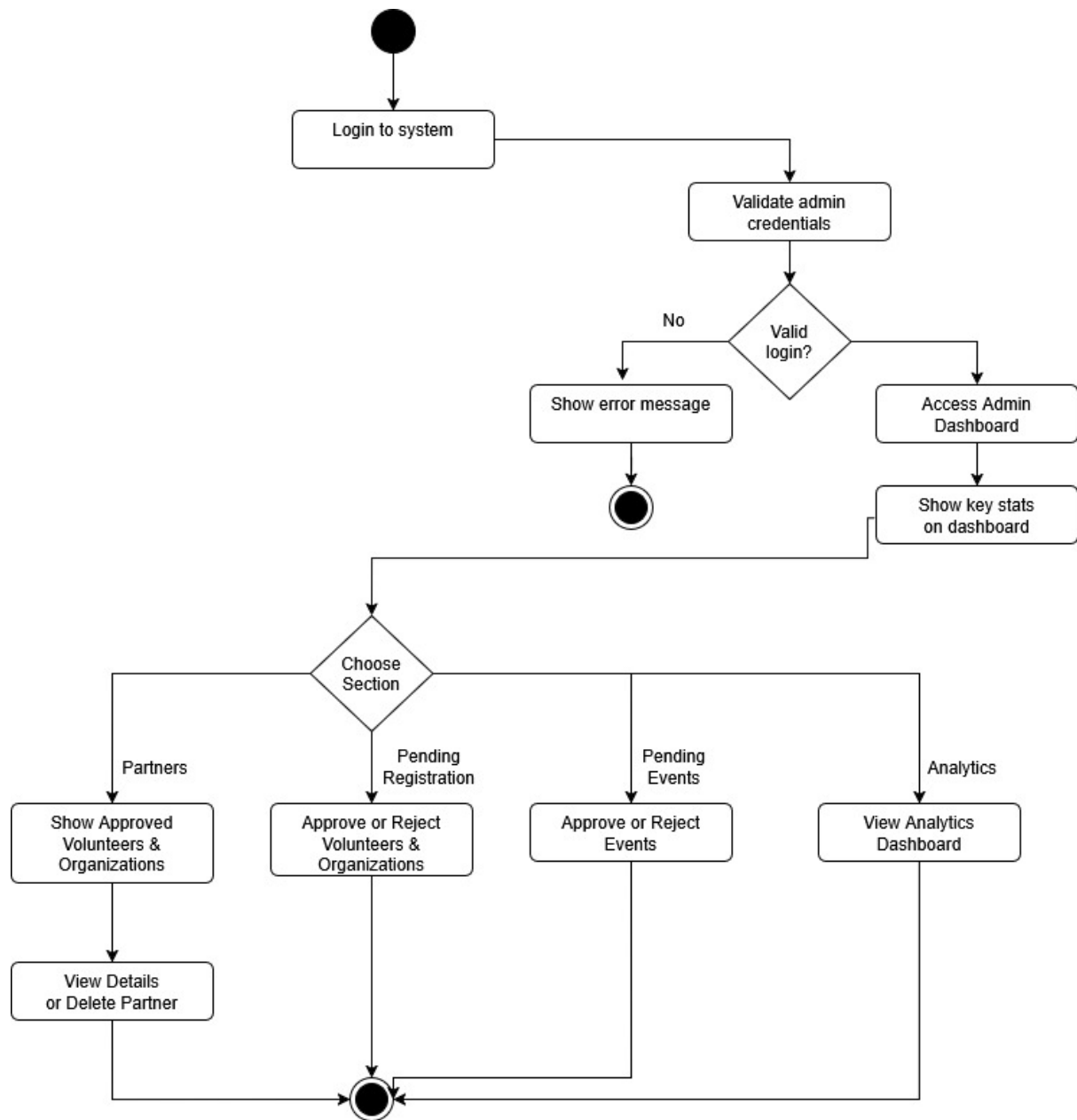


Figure 6: Activity diagram for Admin Module

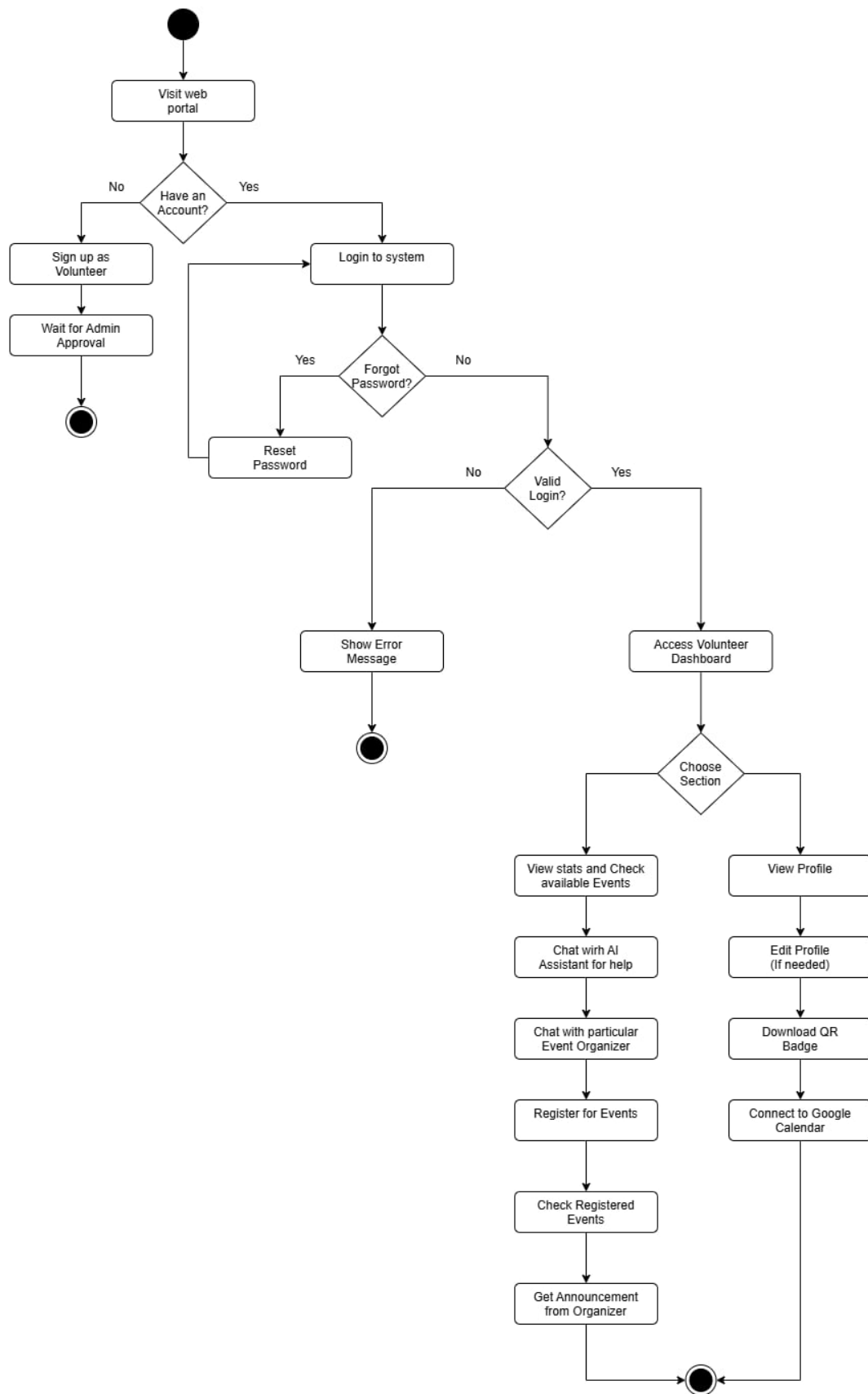


Figure 7: Activity diagram for Volunteer Module

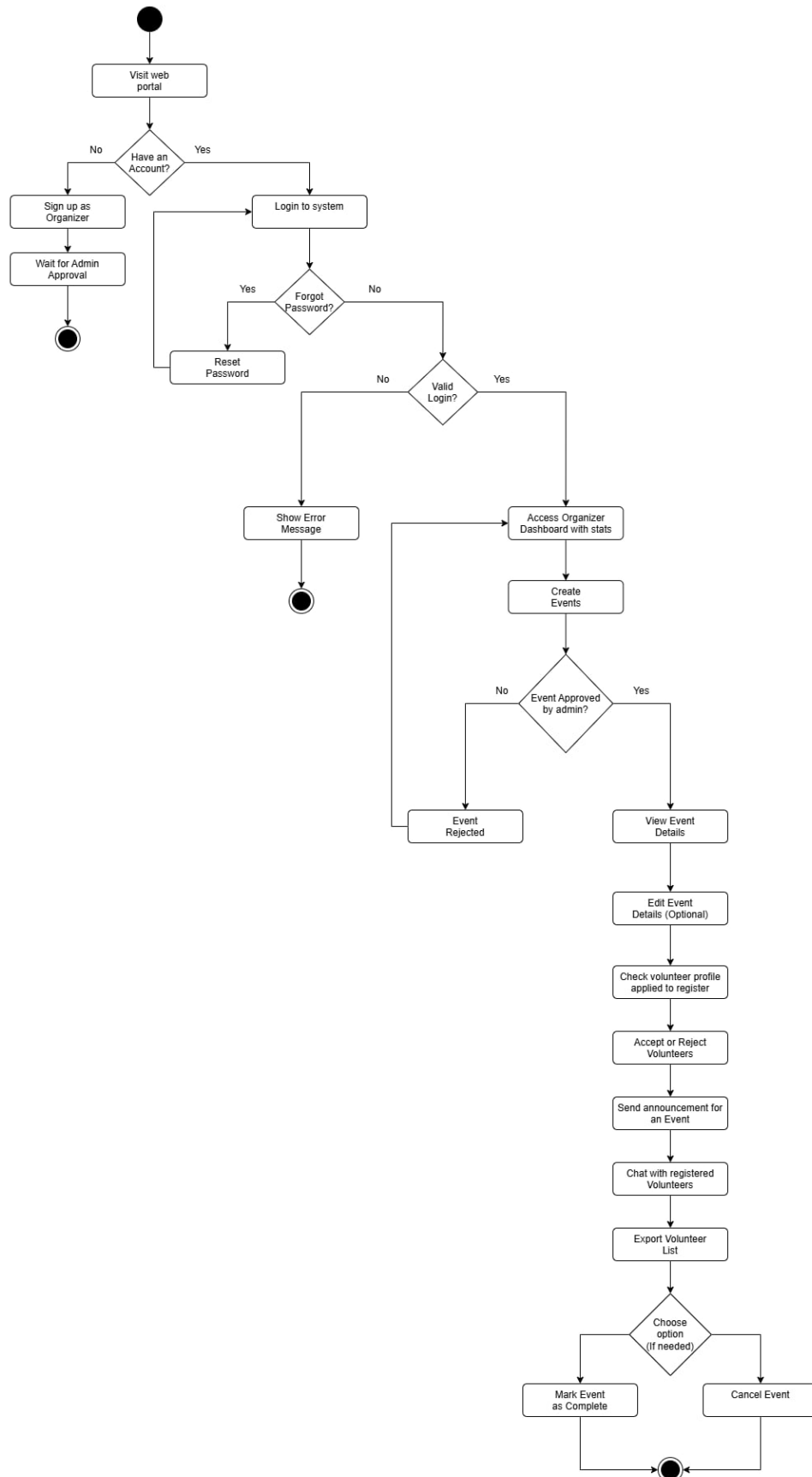


Figure 8: Activity diagram for Organizer Module

9 Appendix

Table 2 presents a structured comparison of the reviewed literature and systems against the key functional areas of Local Volunteer Radar.

Table 2: Literature review and comparison against Local Volunteer Radar

Ref	Title	Findings	Methodology	Limitation
[1]	VolunteerMatch: Online Platform	The platform successfully connects volunteers through profile filtering and keyword search but relies heavily on self-reported information	The platform was evaluated based on profile filtering and keyword search mechanisms	The system lacks role separation, approval workflows, and lifecycle management
[2]	Idealist.org: Nonprofit Portal	The platform centralises nonprofit job and volunteer listings across multiple organisations	The study analysed centralised listings and organisation-managed application handling	There is no built-in approval process or administrative analytics layer

Ref	Title	Findings	Methodology	Gap
[3]	All for Good / Points of Light	Aggregated listings from multiple sources improve opportunity discovery through map-based filtering	The platform was evaluated for aggregated listings and map-based filtering	It does not support unified lifecycle management or structured skill tagging
[4]	Collaborative Filtering (2019)	The recommendation model improves matching accuracy using historical volunteer participation data	Collaborative filtering was applied using historical participation data	The approach excludes structured skill declarations and geospatial filtering
[5]	GIS-Based Location Matching (2020)	GIS-based matching improves volunteer allocation by considering geographic proximity and skill categories	Geographic information systems were used for radius-based and skill classification matching	The model is limited to emergency contexts and lacks lifecycle and administrative controls

10 Conclusion

Local Volunteer Radar equips users with all necessary tools for seamless volunteering. The web application can be accessed at [Render.com](https://render.com) [11], where it is hosted. The publicly available repository, the user manual and this software requirements specification (SRS) document can

be found on GitHub [12].

11 References

- [1] VolunteerMatch, “VolunteerMatch: Online platform.” [Online]. Available: <https://www.volunteermatch.org/>
- [2] Idealist.org, “Idealist.org: Nonprofit portal.” [Online]. Available: <https://www.ideal-ist.org/>
- [3] Points of Light, “All for good.” [Online]. Available: <https://allforgood.org/>
- [4] Y. Zhang and X. Chen, “A collaborative filtering algorithm for volunteer recommendation based on user participation patterns,” in *Proc. Int. Conf. Mach. Learn. Intell. Commun. (MLICOM)*, 2019, pp. 112–123.
- [5] M. A. Rahman and M. S. Islam, “GIS-based volunteer location matching for emergency response,” *Int. J. Disaster Risk Reduction*, vol. 45, pp. 101–112, 2020.
- [6] S. Lee and J. Kim, “Role-based user management in community service platforms,” *J. Inf. Sci.*, vol. 47, no. 3, pp. 321–335, 2021.
- [7] L. Chen and H. Wang, “Calendar integration for event scheduling in volunteer systems,” *IEEE Trans. Serv. Comput.*, vol. 14, no. 5, pp. 1432–1445, Sep./Oct. 2021.
- [8] R. Gupta and A. Sharma, “AI-powered chatbots for user engagement in social platforms,” in *Proc. 2022 IEEE Int. Conf. Artif. Intell. (ICAI)*, 2022, pp. 55–60.
- [9] T. Nakamura and K. Tanaka, “Design of admin dashboards for monitoring community platforms,” *Int. J. Web Inf. Syst.*, vol. 18, no. 2, pp. 89–104, 2022.
- [10] P. Kumar and R. Singh, “Event approval workflows in online community systems,” in *Proc. 2023 15th Int. Conf. Commun. Softw. Netw. (ICCSN)*, 2023, pp. 201–206.

- [11] “Local Volunteer Radar,” Local Volunteer Radar. [Online]. Available: <http://local-volunteer-radar-f.onrender.com> . [Accessed: Apr. 3, 2026].
- [12] “Local Volunteer Radar,” GitHub. [Online]. Available: <https://github.com/ADIB3883/Local-Volunteer-Radar> . [Accessed: Apr. 3, 2026].